

원장 일관성 보장 상태 전이 가드와 멍등 이벤트 처리

역전이 문제 · VALID_TRANSITIONS 맵 · InvalidStateTransitionError
필드 유효성 가드 · 두 상태머신 직렬 관계 · recordProcessedEvent 선점 패턴

LedgerService

VALID_TRANSITIONS

Idempotency

이번 세션의 위치 — S23 다음 단계

원장 데이터 모델이 있다고 원장 일관성이 자동으로 보장되지 않는다

S23에서 배운 것: 왜 오프체인 원장이 필요한가. 4개 테이블 설계 (`mint_requests`, `processed_events`, `user_nft_holdings`, `audit_log`). `ON CONFLICT DO NOTHING` 패턴.

S24에서 배울 것: 분산 환경에서 이벤트 순서가 뒤바뀌면 원장이 어떻게 망가지는가. 이를 막는 3가지 가드 (존재·전이·필드). 멍등 이벤트 처리에서 순서가 왜 중요한가.

Phase 1 맥락 — Webhook 트리거

TX 상태머신의 상태 전이는 월렛원 Webhook이 트리거

Webhook → 내부망 WebhookReceiver → Redis → Consumer →

`LedgerService.transition()`

핵심 질문

- 여러 Consumer Worker가 동시에 같은 이벤트를 처리하면?
- 네트워크 재전달로 이미 처리한 이벤트가 다시 오면?
- 이벤트 순서가 보장되지 않으면?

분산 시스템 — 메시지 순서는 보장되지 않는다

Consumer Group Worker 2개 인스턴스, 같은 `mint_request`에 대한 이벤트가 비슷한 시간에 도착

정상 처리 순서

- 1 **message-1** — STATUS_SUBMITTED (txHash=0xABC)
 - 2 **message-2** — STATUS_MINED (blockNumber=12345)
 - 3 **message-3** — STATUS_FINALIZED
- A **Worker-A** — message-1 → REQUESTED→SUBMITTED ✓
- B **Worker-B** — message-2 → SUBMITTED→MINED ✓
- A **Worker-A** — message-3 → MINED→FINALIZED ✓

실제 발생 가능 (Worker-B 일시 지연)

- 1 **Worker-A** — message-1 → REQUESTED→SUBMITTED
- ! **Worker-A** — message-3 → SUBMITTED→FINALIZED (message-2 건너뛴!)
- ? **Worker-B** — message-2 → "FINALIZED→MINED 전이하라" ??

또는 네트워크 재전달:

message-2 ACK 전에 Worker 재시작 → Redis가 message-2 재전달
→ 이미 FINALIZED인데 MINED로 되돌아가라는 이벤트 도착

역전이(backward transition): 이미 finality 확보된 TX가 "아직 블록에만 있음"으로 원장에 표시됨. ReconcileService가 잘못된 불일치를 감지하고 알람 발생 → 담당자 혼란, 감사 추적 파괴.

가드가 없으면 어떻게 되나

두 가지 역전이 시나리오의 실제 결과

시나리오 A: FINALIZED → MINED 역전이

- 1 현재 상태: **FINALIZED**
- ! "MINED로 전이하라"는 이벤트 도착
- 2 원장이 FINALIZED → MINED로 되돌아감
- 3 이미 finality 확보된 TX가 "아직 블록에만 있음"으로 원장에 표시됨
- 4 ReconcileService가 잘못된 불일치 감지 → 담당자 혼란, 감사 추적 파괴

금융 시스템에서 한 번 진행된 거래가 이전 상태로 되돌아가는 것은 심각한 데이터 불일치

시나리오 B: 중단 상태(FAILED) 이후 전이

- 1 현재 상태: **FAILED**
- ! 어떤 이벤트로 인해 CONFIRMED로 전이 시도
- 2 이미 실패 처리된 발행 요청이 "확인됨" 상태로 바뀜
- 3 user_nft_holdings에 존재하지 않는 토큰 추가 → 사용자가 없는 NFT를 가진 것처럼 보임
- 4 감사 추적과 실제 온체인 상태가 불일치

핵심 설계 원칙: "어떤 상태에서 어떤 상태로의 전이가 허용되는가"를 코드에 명시한다 → **VALID_TRANSITIONS 맵**

VALID_TRANSITIONS 맵 — 허용 전이 화이트리스트

M3 S13 TxStateMachineService와 동일한 패턴 — LedgerService에도 독립 적용

```
// LedgerService.ts
private static readonly VALID_TRANSITIONS: Record<MintStatus, MintStatus[]> = {
  REQUESTED: ['SUBMITTED', 'FAILED'],           // 발행 요청 생성 → VASP 제출 or 즉시 실패
  SUBMITTED: ['MINED', 'FAILED'],                // TX 제출 완료 → 블록 포함 or 실패
  MINED: ['CONFIRMED', 'REORGED', 'FAILED'],     // 블록 포함 → 충분한 확인 or 재편 or 실패
  CONFIRMED: ['FINALIZED'],                     // 충분한 블록 확인 → PoS Finality 대기
  FINALIZED: [],                                // 종단 상태 — PoS 절대 불변, 이후 전이 없음
  FAILED: [],                                   // 종단 상태 — 재발행하려면 새 요청 필요
  REORGED: ['MINED', 'FAILED'],                  // 재편 → MINED 복귀(confirmation 재시작) or 포기
};
```

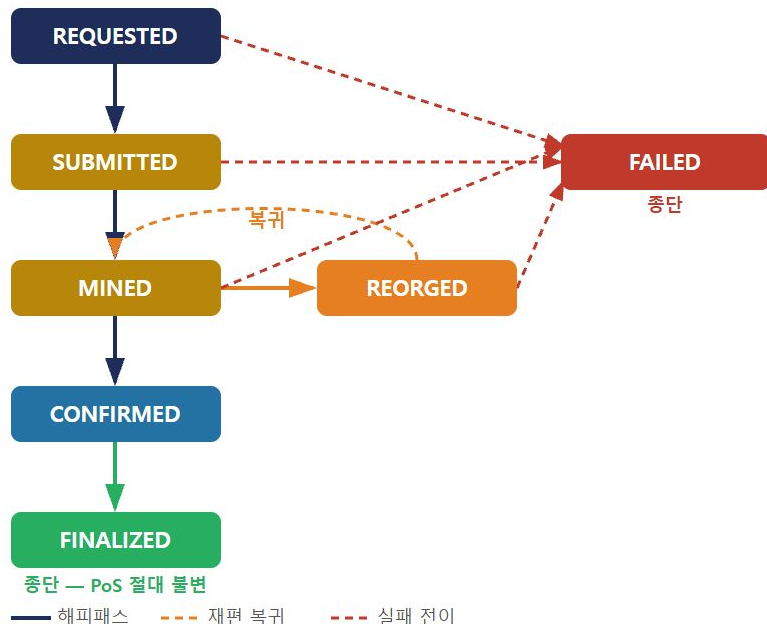
왜 두 레이어가 독립적으로 가져야 하나?

TxStateMachineService가 FINALIZED 전이 완료 → 이벤트 → LedgerService
FINALIZED 전이 트리거. 두 레이어가 각자 VALID_TRANSITIONS 가드를 갖고 있
어야 어느 쪽에서 중복 이벤트가 와도 안전.

종단 상태(**terminal state**): FINALIZED와 FAILED는 허용 전이 목록이 []. 어떤
전이 시도도 즉시 `InvalidStateTransitionError` 로 차단.

MintStatus 상태 전이 다이어그램 + 상태별 의미

7개 상태 · 2개 중단 상태(FINALIZED / FAILED)



상태	의미	설정 주체	txHash
REQUESTED	발행 요청 생성	IssuerService	NULL
SUBMITTED	VASP에 TX 제출, 블록 대기	TxStateMachine	있음
MINED	블록 포함, REORG 가능 구간	TxStateMachine	있음
CONFIRMED	충분한 블록 확인	TxStateMachine	있음
FINALIZED	PoS 2/3+ validator 동의 → 불변	TxStateMachine	있음
FAILED	TX 실패, 재발행 시 새 요청	TxStateMachine	있을 수도
REORGED	체인 재편으로 TX 소실(임시)	TxStateMachine	있음(무효)

두 상태머신의 관계 — TxStateMachineService vs LedgerService

M3 외부 상태 동기화 + M4 내부 원장 일관성 — 독립적이지만 직렬로 작동

구분	TxStateMachineService (M3)	LedgerService (M4)
추적 대상	VASP/블록체인의 TX 상태	내부 발행 요청의 원장 상태
상태 타입	TxStatus (8개, PENDING·REORGED 포함)	MintStatus (7개)
위치	내부망 (VASP 통신 레이어)	Core-Banking (원장 레이어)
트리거	VASP 콜백, 블록 이벤트, 폴링	TxStateMachineService 호출
역할	외부 상태 동기화	내부 원장 일관성 보장
실패 시	재시도 (gas bump, REORG 복구)	InvalidStateTransitionError 예외
VALID_TRANSITIONS 필요 이유	블록체인 이벤트 순서 보장 안 됨	Consumer Worker 중복·순서 역전

M3 없이 M4가 혼자 동작하지 않는다: M3 상태머신이 먼저 외부(체인) 상태를 동기화하고, 그 결과가 M4 원장 상태 전이를 트리거한다.

왜 두 개의 상태머신인가 — 추적 대상이 다르다

TxStateMachineService = "블록체인 TX가 어디까지 갔는가" · LedgerService = "내부 원장 기록이 어떤 상태인가"

핵심 이유 — 둘은 어긋날 수 있다 (의도적)

TxStateMachine: REQUESTED → SUBMITTED → MINED → CONFIRMED →

FINALIZED ✓

LedgerService : REQUESTED → SUBMITTED → **MINED** (이벤트 아직 미도달)

이 불일치를 **ReconcileService**가 감지한다. 하나로 합쳤다면 "어긋난 상태"라는 개념 자체가 없어진다.

보호 대상이 다르다

	TxStateMachineService	LedgerService
위험	REORG · mempool 지연 · gas 부족	Consumer Worker 중복 · 순서 역전
방어	gas bump · REORG 대기 · 폴링	InvalidStateTransitionError · processed_events
실패시	재시도 / TX 재전송	조용히 reject

하나로 합쳤다면

- ✗ Consumer Worker 중복 이벤트에 대한 원장 레이어 독립 가드 없음
- ✗ VASP 통신 로직과 원장 일관성 코드가 한 클래스에 혼재
- ✗ 블록체인-원장 Gap 자체가 사라져 이벤트 유실·지연을 감지할 수 없음 (대사 불가)

블록체인 상태와 내부 원장 상태는 **항상 동기**라는 보장이 없다.

그 갭을 **의도적으로 허용**하고 모니터링해야 한다.

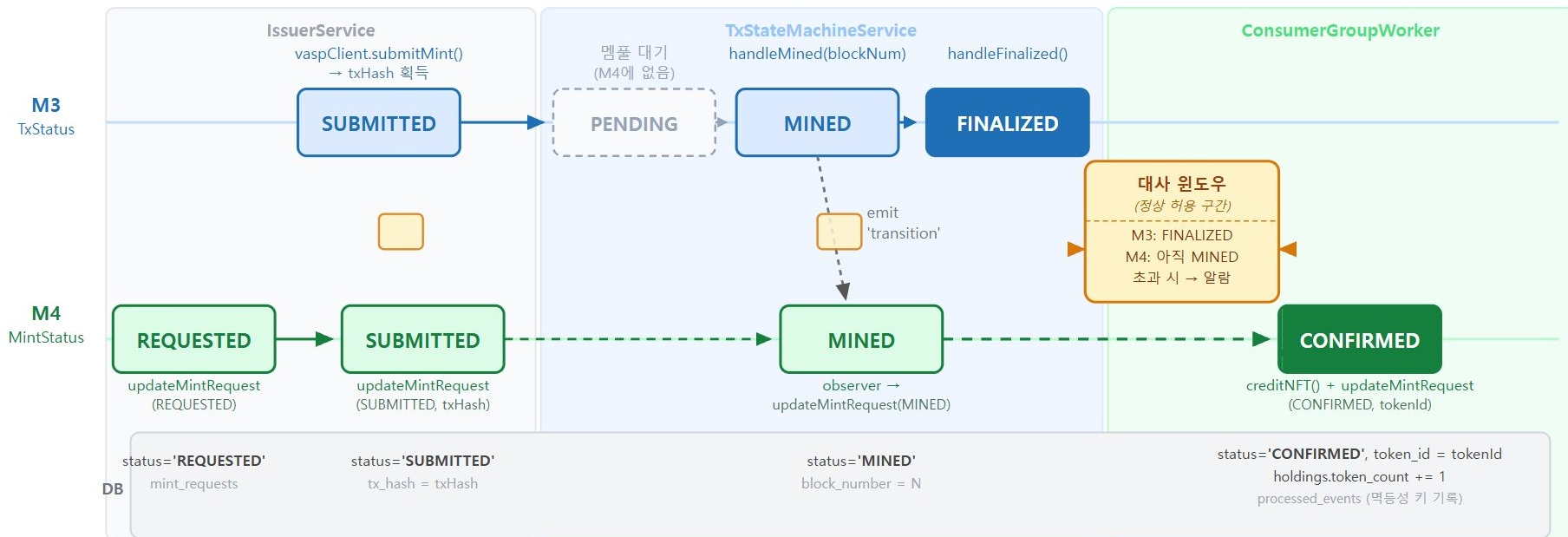
분리해야만 불일치를 감지할 수 있다.

Observer 패턴을 쓰는 이유도 여기에 있다: TxStateMachineService가

LedgerService를 직접 호출하지 않고 이벤트를 방출하는 것은 — 결합을 끊어야 두 레이어를 독립적으로 테스트·모니터링할 수 있기 때문

두 상태머신 직렬 작동 — 전체 발행 흐름

같은 체인 이벤트를 M3·M4가 다른 시점에 반영 — 정합이 어긋나는 구간이 ReconcileService 대사 윈도우



※ 비정상 알람 시나리오: M3=REORGED + M4=CONFIRMED (원장 롤백 필요) · M3=FAILED + M4 미반영 · Observer 이벤트 유실로 M4=SUBMITTED 고착 · 허용 시간 초과

_transition() + EventEmitter Observer 패턴

모든 상태 전이는 이 **private** 메서드를 통과 · 'transition' 이벤트 방출 → 느슨한 결합

```
/** 상태 전이 + Observer 이벤트 방출 */
private async _transition(
  req: MintRequest,
  to: TxStatus,
  extra?: Partial<MintRequest>,
): Promise<void> {
  const from = req.status;
  // 1. 전이 유효성 검사
  if (!VALID_TRANSITIONS[from].includes(to)) {
    throw new InvalidStatusTransitionError(from, to);
  }
  // 2. DB 업데이트
  await this.repo.updateStatus(req.id, to, extra);
  // 3. Observer 이벤트 방출
  const updated: MintRequest = {
    ...req, status: to, ...extra,
    updatedAt: new Date()
  };
  this.emit('transition', {
    requestId: req.id, from, to, req: updated,
  } satisfies TxTransitionEvent);
}
```

Observer 패턴 (EventEmitter):

TxStateMachineService가 직접 LedgerService를 호출하지 않는다.
'transition' 이벤트를 방출 → 구독자(ConsumerGroupWorker)가 반응

```
// 구독 방법 (호출 측)
txService.on('transition',
  (e: TxTransitionEvent) => {
    if (e.to === 'CONFIRMED') {
      ledger.recordHolding(e.req);
    }
    if (e.to === 'FAILED') {
      alertOps(e.req);
    }
  }
);
```

satisfies 키워드 (TS 4.9+):

`satisfies TxTransitionEvent` = 타입 확인은 하되 추론 유지.
emit() 인자가 TxTransitionEvent 형태임을 컴파일 타임에 검증.

InvalidStateTransitionError — 허용되지 않은 상태 전이

VALID_TRANSITIONS 가드를 통과하지 못한 전이 시도 시 발생

// 허용되지 않은 상태 전이

```
export class InvalidStateTransitionError extends Error {  
  constructor(from: MintStatus, to: MintStatus) {  
    super(`Invalid state transition: ${from} → ${to}`);  
    this.name = 'InvalidStateTransitionError';  
  }  
}
```

에러 이름(name)을 명시적으로 설정하는 이유:

TypeScript에서 `instanceof` 가 클래스 이름으로 판별.

catch 블록에서 `err instanceof InvalidStateTransitionError` 로 구분 — 중복 이벤트는 조용히 무시, DLQ로 넘기지 않음.

발생 조건: `VALID_TRANSITIONS[from]` 에 `to` 가 포함되지 않은 경우

→ DB UPDATE 실행 없이 즉시 throw

동작 예시

예시 1: CONFIRMED → SUBMITTED 시도 (역방향)

`allowed = VALID_TRANSITIONS['CONFIRMED'] = []`

'SUBMITTED'이 [] 안에 없음

→ `throw InvalidStateTransitionError('CONFIRMED', 'SUBMITTED')`

→ DB UPDATE 실행 안 됨

예시 2: MINED → CONFIRMED 시도 (정상)

`allowed = VALID_TRANSITIONS['MINED']`

`= ['CONFIRMED', 'REORGED', 'FAILED']`

'CONFIRMED'이 목록 안에 있음 → 가드 통과

→ 다음 단계(필드 유효성 가드)로 진행

예시 3: FAILED → 어떤 상태든 (종단 탈출 시도)

`allowed = VALID_TRANSITIONS['FAILED'] = []`

→ 항상 `InvalidStateTransitionError`

→ FAILED는 종단 — 재시도 불가

MintRequestNotFoundError — 존재하지 않는 레코드

상태 전이 전 DB 조회 시 레코드가 없을 때 발생 — 전이 가드보다 먼저 실행

```
// 레코드가 DB에 없을 때
export class MintRequestNotFoundError extends Error {
  constructor(requestId: string) {
    super(`MintRequest not found: ${requestId}`);
    this.name = 'MintRequestNotFoundError';
  }
}
```

발생 순서: updateMintRequest() 호출 시

- ① getMintRequest(id) → null이면 즉시 throw
- ② null이 아니면 → InvalidStateTransitionError 가드로 이동

운영 함의: MintRequestNotFoundError는 정상 중복 이벤트와 다르다.

중복 이벤트 → processedEvents 에서 차단 (더 앞 단계).

NotFound → 레코드 자체 소실 — 운영자 확인 필요.

동작 예시

예시 1: 존재하지 않는 requestId로 전이 시도

updateMintRequest('non-existent-id', { status: 'SUBMITTED' })

- getMintRequest('non-existent-id') 조회
- DB row 없음 → null 반환
- throw MintRequestNotFoundError('non-existent-id')
- DB UPDATE 실행 안 됨
- InvalidStateTransitionError 가드까지 도달 안 함

예시 2: 30일 만료로 자동 삭제된 레코드

CONFIRMED/FAILED 후 30일 경과 → pg_cron이 row 삭제

→ 뒤늦은 FINALIZED 이벤트 수신 시 조회 → null

→ MintRequestNotFoundError

→ 운영자 알림 필요 (재발행 불가)

updateMintRequest() — 3단계 가드 전체 구현

레코드 존재 확인 → 상태 전이 가드 → 필드 유효성 가드 → DB UPDATE → Audit log

```
async updateMintRequest(
  requestId: string,
  patch: { status: MintStatus; txHash?: string;
          tokenId?: bigint; errorMsg?: string },
): Promise<MintRequest> {
  // 1. 레코드 존재 확인
  const current = await this.getMintRequest(requestId);
  if (!current) throw new MintRequestNotFoundError(requestId);

  // 2. 상태 전이 가드
  const allowed = LedgerService.VALID_TRANSITIONS[current.status];
  if (!allowed.includes(patch.status))
    throw new InvalidStateTransitionError(current.status, patch.status);

  // 3. 필드 유효성 가드
  if (patch.status === 'SUBMITTED' && !patch.txHash)
    throw new Error('txHash is required for SUBMITTED');
  if (patch.status === 'CONFIRMED' && !patch.tokenId)
    throw new Error('tokenId is required for CONFIRMED');
```

```
// 4. DB UPDATE - COALESCE로 기존 값 보존
await this.db.query(
  `UPDATE mint_requests
   SET status      = $1,
       tx_hash     = COALESCE($2, tx_hash),
       token_id    = COALESCE($3, token_id),
       error_msg   = $4,
       updated_at  = NOW()
  WHERE request_id = $5`,
  [patch.status,
   patch.txHash ?? null,
   patch.tokenId?.toString() ?? null,
   patch.errorMsg ?? null,
   requestId],
);

// 5. Audit log - 상태 변경 기록
const afterState = { ...current, ...patch };
await this.auditLog.log({
  actor: 'system', action: `STATUS_${patch.status}`,
  resourceType: 'MintRequest', resourceId: requestId,
  beforeState: current, afterState,
});
return afterState as MintRequest;
}
```

왜 필드 유효성 가드도 필요한가 — 좀비 레코드

상태 전이 가드만으로는 충분하지 않다 — SUBMITTED + txHash 없음 = 폴링 불가 좀비

문제 시나리오

```
// 이걸 상태 전이 가드를 통과한다!  
await ledger.updateMintRequest(requestId, {  
  status: 'SUBMITTED',  
  txHash: undefined, // txHash 없이 SUBMITTED?  
});  
  
// 결과: mint_requests.tx_hash = NULL  
// SUBMITTED 상태인데 txHash가 없음  
// → TxStateMachineService가 어떤 TX를 폴링해야 하는지 모름  
// → pollStaleRequests: tx_hash IS NULL → 무시  
// → SUBMITTED 상태에 영원히 머무르는 좀비 레코드
```

두 가드의 역할 분리

가드 종류 체크 대상

상태 전이 가드	"어떤 상태에서 어떤 상태로 가는지"
필드 유효성 가드	"그 상태로 전이할 때 필요한 데이터가 있는지"

상태 필수 필드 없으면

SUBMITTED	txHash	폴링 불가 좀비 레코드
MINED	txHash	확인 대상 TX 알 수 없음
CONFIRMED	tokenId	holdings 연결 불가

두 가드가 함께 있어야 완전한 원장 일관성 보장

먹등 이벤트 처리 — 중복 이벤트 도착 시나리오

Redis Streams at-least-once delivery → 같은 이벤트가 두 번 도착하는 시나리오

중복 이벤트 발생 시나리오

- 1 t=0: **Worker-A**가 NFTIssued 이벤트 처리 시작
- 2 t=1: mint_requests UPDATE (CONFIRMED) ✓
- 3 t=2: user_nft_holdings INSERT ✓
- ! t=3: **Worker-A**가 XACK 전에 프로세스 종료 ← 장애!
- 4 t=4: Redis — XACK 없음 → PEL에 메시지 남음
- 5 t=5: **Worker-B**가 PEL 재처리 → 같은 이벤트 다시 처리 시작

가드만 있는 경우의 처리

- 6 t=6: mint_requests UPDATE (CONFIRMED) 시도 → 이미 CONFIRMED인 상태
- ! VALID_TRANSITIONS['CONFIRMED'] = [] → InvalidStateTransitionError 발생!
- 7 Worker-B가 에러 → DLQ로 이동? → 알림 발생 → 담당자 혼란

더 나은 방법: 상태 전이 가드가 막아주는 것은 하지만, 이미 처리된 이벤트임을 먼저 판별하는 게 더 깔끔하다. 이게 바로 **M2에서 배운 먹등성 가드** —

`processed_events` 테이블에 이벤트 ID를 기록해 중복 수신 시 조용히 skip.

IdempotencyGuard vs recordProcessedEvent()

같은 목적, 다른 저장소 — 레이어 특성에 따라 선택

IdempotencyGuard (Redis / InMemory)

```
// M2에서 배운 방식
const processed = await idempotency.run(
  `NFTIssued:${requestId}`,
  async () => {
    await ledger.creditNFT(to, tokenId);
  }
);
if (!processed) logger.info('duplicate skipped');
```

장점: 빠르다 — Redis 조회 1회로 끝

단점: Redis 재시작 시 기억 소실 · payload/블록 정보 없음

적합한 레이어: 웹훅-API 수신 — 빠른 중복 차단이 우선, 장기 추적 불필요

recordProcessedEvent() (PostgreSQL)

```
// M4에서 배우는 방식
const result = await ledger.recordProcessedEvent(
  txHash, logIndex, 'NFTIssued', blockNumber, payload
);
if (result.skipped) return; // 중복 — 조용히 skip
// 처음 수신 → 이후 처리 진행
```

장점: 재시작 후에도 영구 보존 · tx_hash: block_number: payload 기록
추가 용도: 90일 보존 → Reorg 감지, 감사 추적 가능

적합한 레이어: 블록체인 이벤트 원장 — "빠른 skip"보다 "정확한 기록"이 우선

결론: 둘 다 ON CONFLICT DO NOTHING 원리로 중복을 막는다. 웹훅-API는 Redis, 블록체인 원장은 DB — 레이어 특성에 맞게 선택.

recordProcessedEvent가 첫 번째 줄이어야 하는 이유

선점(sentinel) 패턴 — 처리 시작을 DB에 먼저 기록

잘못된 순서

```
// 잘못된 순서
1. updateMintRequest(CONFIRMED) 실행 ✓
2. addHolding 실행 ✓
3. recordProcessedEvent ← 여기서 중복 체크?

// 문제: 1+2 완료 후 Worker 죽으면,
// 재처리 시 recordProcessedEvent가 첫 번째여도
// → skipped: false (아직 processed_events에 없음)
// → updateMintRequest(CONFIRMED) 또 시도
// → 이미 CONFIRMED → InvalidStateTransitionError!
// → DLQ로 이동, 담당자 알림
```

올바른 순서

```
async handleNFTIssued(event) {
  // 1단계: 중복 이벤트 체크 — 반드시 가장 먼저
  // "이 이벤트를 처리 중"임을 DB에 선점 기록
  const evResult = await ledger.recordProcessedEvent(
    event.txHash, event.logIndex,
    'NFTIssued', event.blockNumber, event.payload,
  );
  if (evResult.skipped) {
    return; // 이미 처리됨 → 조용히 XACK하고 종료
  }
  // 2단계: 신규 이벤트만 비즈니스 로직 실행
  await ledger.updateMintRequest(requestId, {
    status: 'CONFIRMED', tokenId: event.tokenId,
  });
  await ledger.addHolding(userId, event.tokenId, policyId);
}
```

recordProcessedEvent 구현 — ON CONFLICT DO NOTHING

UNIQUE(tx_hash, log_index) + RETURNING id → rows.length로 신규/중복 판별

```
async recordProcessedEvent(  
  txHash: string,  
  logIndex: number,  
  eventName: string,  
  blockNumber: bigint,  
  payload: unknown,  
) : Promise<ProcessedEventResult> {  
  const result = await this.db.query(  
    `INSERT INTO processed_events  
      (tx_hash, log_index, event_name,  
       block_number, payload)  
    VALUES ($1, $2, $3, $4, $5)  
    ON CONFLICT (tx_hash, log_index) DO NOTHING  
    RETURNING id`,  
    [txHash, logIndex, eventName,  
     blockNumber.toString(), JSON.stringify(payload)],  
  );  
  if (result.rows.length === 0) {  
    return { skipped: true }; // UNIQUE 충돌 - 이미 처리됨  
  }  
  return { skipped: false, id: result.rows[0].id as number };  
}
```

ON CONFLICT DO NOTHING 동작:

UNIQUE(tx_hash, log_index) 충돌 시 → 아무것도 INSERT하지 않음 →
RETURNING에서 행 반환 없음 → rows.length === 0 → skipped: true

같은 txHash, 다른 logIndex:

하나의 TX에 여러 이벤트 로그가 있을 수 있음. (txHash, logIndex) 쌍이 이벤트의 유일 식별자. 같은 txHash라도 logIndex가 다르면 별개 이벤트 → 각각
skipped: false

SELECT-then-INSERT는 안전하지 않다: 동시에 두 Worker가 SELECT → 둘 다 "없음"으로 판단 → 둘 다 INSERT 시도 → 중복 발행. ON CONFLICT가 원자적으로 처리.

실습 테스트 케이스 전체

LedgerService 상태 전이 가드 + recordProcessedEvent 8가지 케이스

상태 전이 가드 테스트

케이스	기대 결과
REQUESTED → SUBMITTED (txHash 있음)	통과
CONFIRMED → SUBMITTED	InvalidStateTransitionError
FAILED → CONFIRMED	InvalidStateTransitionError
MINED → CONFIRMED (tokenId 없음)	InvalidStateTransitionError
SUBMITTED + txHash 누락	txHash is required
REORGED → MINED 복귀	통과

recordProcessedEvent 테스트

케이스	기대 결과
동일 txHash+logIndex 2회	2번째 skipped: true
같은 txHash, 다른 logIndex	둘 다 skipped: false

구현 체크리스트:

- ① VALID_TRANSITIONS에 7개 상태 모두 정의
- ② updateMintRequest 3단계 가드 순서
- ③ COALESCE(\$2, tx_hash) — 기존 값 보존
- ④ auditLog.log() 상태 전이마다 호출
- ⑤ recordProcessedEvent가 핸들러 첫 줄

S24 완료 기준

상태 전이 가드 구현 + 먹등 이벤트 처리 이해 확인

상태 전이 가드

- ✓ REQUESTED→SUBMITTED→MINED→FINALIZED→CONFIRMED 정방향 모두 통과
- ✓ CONFIRMED→SUBMITTED, FAILED→CONFIRMED 등 역전이 → InvalidStateTransitionError
- ✓ MINED→CONFIRMED 차단 (CONFIRMED는 MINED에서 직접 올 수 없음 — FINALIZED 필수 경유)
- ✓ SUBMITTED 전이 시 txHash 누락 → 에러
- ✓ CONFIRMED 전이 시 tokenId 누락 → 에러
- ✓ REORGED → MINED 복귀 정상 통과

먹등 이벤트 처리

- ✓ 동일 (txHash, logIndex) 2회 → skipped: true
- ✓ 같은 txHash, 다른 logIndex → 각각 skipped: false
- ✓ recordProcessedEvent가 이벤트 핸들러의 첫 번째 줄이어야 하는 이유 설명 가능
- ✓ 두 상태머신(TxStateMachineService / LedgerService) 역할 차이 설명 가능
- ✓ COALESCE(\$2, tx_hash) 패턴이 필요한 이유 설명 가능